

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 9

Watching System Calls with strace

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. System calls of a real command **2**
2.1 Observations 2

3 Part 2. A program that calls the kernel directly **2**
3.1 Observations 4

4 Part 3. Counting system calls with strace **4**
4.1 Counting write system calls in `./syscalls` 4
4.2 Counting openat system calls in `ls -l` 5

5 Conclusion **5**

1 Goal of the work

The primary objective of this laboratory assignment is to observe the boundary between a user program and the operating system kernel, the system call. Using `strace`, a tool that prints every system call a process makes, I examined the system calls made by a standard command (`ls -l`) and by a small C program that uses `open`, `write` and `read` directly. The work was performed in an Ubuntu 24.04 (arm64) container running on Docker Desktop on macOS, with strace 6.8 and gcc 13.3.0.

2 Part 1. System calls of a real command

I ran `ls -l` under `strace`, with the trace filtered to the calls relevant to reading and printing:

```
root@ad1991a5683f:/work# strace -f -e trace=openat,read,write,close ls -l
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/aarch64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tpr"..., 1024) = 566
read(3, "", 1024)                       = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
write(1, "total 0\n", 8total 0
)                = 8
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Most of the calls come from the dynamic loader and the C library, which open shared libraries and read `/proc/filesystems` while the program starts. The directory itself is opened with `openat` on `"."`. The directory entries are read with `getdents64`, which is not part of the filter and therefore does not appear above.

2.1 Observations

1. **Printing to the screen:** `ls` uses the `write` system call to print its output. The line `write(1, "total 0", 8)` shows that the output goes to file descriptor **1**, standard output. The directory was empty at the time of the run, so the listing is only the `total 0` line.

2. **Number of calls:** The trace above, which is filtered, contains about 20 calls. The total number of system calls made by `ls` is larger, because the filter hides calls such as `getdents64`, `mmap` and `fstat`.

3 Part 2. A program that calls the kernel directly

I wrote `syscalls.c`, a program that opens a file, writes one line to it, closes it, then reopens it and reads the line back. It uses the system calls `open`, `write` and `read` directly for the file operations; `printf` is used only to show the results on the screen.

```c
#include <fcntl.h>  /* open */
#include <unistd.h> /* write, read, close */
#include <string.h> /* strlen */
#include <stdio.h>  /* printf */

int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor
     */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

The program was compiled and run as follows. The input is empty, because the program takes no command-line arguments.

```
root@ad1991a5683f:/work# gcc -Wall -Wextra -o syscalls syscalls.c
root@ad1991a5683f:/work# ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
```

Next, the same program was traced with strace, showing only the calls that correspond to the file operations:

```
root@ad1991a5683f:/work# strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(1, "open returned descriptor 3\n", 27open returned descriptor 3
) = 27
write(3, "hello, kernel\n", 14)          = 14
write(1, "write sent 14 bytes\n", 20write sent 14 bytes
)   = 20
close(3)                                 = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
write(1, "read got 14 bytes: hello, kernel...", 33) = 33
```

```
close(3)                                = 0
+++ exited with 0 +++
```

Each C call maps to one system call in the trace:

C call	System call in the trace	Result
`open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644)`	`openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644)`	descriptor 3
`write(fd, msg, strlen(msg))`	`write(3, "hello, kernel\n", 14)`	14 bytes
`close(fd)`	`close(3)`	0
`open("note.txt", O_RDONLY)`	`openat(AT_FDCWD, "note.txt", O_RDONLY)`	descriptor 3
`read(fd, buf, sizeof(buf) - 1)`	`read(3, "hello, kernel\n", 63)`	14 bytes

The `printf` calls appear in the trace as `write(1, ...)`, which shows that they are also system calls, aimed at standard output.

3.1 Observations

1. **Why the first descriptor is 3:** Descriptors **0** (standard input), **1** (standard output) and **2** (standard error) are opened by the operating system for every process before it starts. The first file the program opens therefore receives the lowest free number, which is 3.

2. **The read size:** The program asks for at most 63 bytes (`sizeof(buf) - 1`), but the file holds only 14, so `read` returns 14. The kernel never returns more data than the file contains.

4 Part 3. Counting system calls with strace

To analyze the frequency and execution time of specific system calls, I used the counting feature of `strace` (`-c`). Specifically, I measured the `write` system calls made by the compiled C program (`./syscalls`) and the `openat` calls made by the `ls -l` command.

4.1 Counting write system calls in `./syscalls`

I ran `strace` with the summary statistics option (`-c`) filtered to the `write` system call:

```
root@5a3be17a0a4a:/work# strace -c -e trace=write ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000010           2         4           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000010           2         4           total
```

Explanation of output: The program's output appears directly before the summary table because the standard output of the executing program mixes with the summary statistics printed by `strace` to standard error upon exit.

The summary table shows that exactly **4** `write` system calls were executed, taking a total of 0.000010 seconds (an average of 2 microseconds per call).

To verify where these 4 calls were directed, running `strace -e trace=write ./syscalls` without the summary flag reveals the exact target file descriptors:

```
write(1, "open returned descriptor 3\n", 27) = 27
write(3, "hello, kernel\n", 14)          = 14
write(1, "write sent 14 bytes\n", 20)    = 20
write(1, "read got 14 bytes: hello, kernel...", 33) = 33
```

Out of the 4 `write` calls, three are sent to file descriptor **1** (standard output, originating from the program's `printf()` informative status messages), and one is sent to file descriptor **3** (the file `note.txt`, opened via `openat()`). The `openat()` system call returns descriptor **3** because descriptors 0, 1, and 2 are already reserved by the operating system for standard input, output, and error.

4.2 Counting openat system calls in `ls -l`

Similarly, I analyzed the `openat` system calls invoked during the execution of `ls -l`:

```
root@5a3be17a0a4a:/work# strace -c -e trace=openat ls -l
total 80
-rw-r--r-- 1 root root    14 Nov  9 12:51 note.txt
-rwxr-xr-x 1 root root 70672 Nov  9 12:51 syscalls
-rw-r--r-- 1 root root  1037 Nov  5 12:03 syscalls.c
% time     seconds  usecs/call    calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.000243          24        10         1 openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000243          24        10         1 total
```

Explanation of output: The summary table indicates that `ls -l` executed a total of **10** `openat` system calls, resulting in **1** error.

The error corresponds to an attempt to open a configuration or localization file (such as `/etc/localtime`) that does not exist in the container environment, returning `-1 ENOENT (No such file or directory)`. The remaining successful `openat` calls were used by the dynamic linker and C library to load shared libraries and check configuration files during startup, as well as opening the current directory ('.').

5 Conclusion

Through this laboratory assignment, I observed the system call interface from the outside. Even a simple command such as `ls` makes many system calls, most of them for loading libraries and reading files, and only the final `write` sends output to the user. The C program showed that familiar file operations are direct requests to the kernel: `open` returns a descriptor, `write` and `read` move data, and `close` releases the descriptor. Furthermore, using `strace -c` allowed quantifying system call performance and identifying specific error conditions (such as missing configuration files during process initialization).